

Working with Basic Controls:

Button Control: The Windows Forms **Button** control allows the user to click it to perform an action. The **Button** control can display both text and images. When the button is clicked, it looks as if it is being pushed in and released.

Properties:

1) Text: Gets or sets the text caption displayed in the Button control.

Syntax : instance.Text = value

Instance: It contains name of the control with which this property is applied.

2) Flatstyle: Gets or sets the flat style appearance of the button control.

Syntax : instance.FlatStyle = *value*

Instance: It contains name of the control with which this property is applied.

The default value is **Standard**.

Member name	Description
-------------	-------------

Flat	The control appears flat.
-------------	---------------------------

Popup	A control appears flat until the mouse pointer moves over it, at which point it appears three-dimensional.
--------------	--

Standard	The control appears three-dimensional.
-----------------	--

System	The appearance of the control is determined by the user's operating system.
---------------	---

3) Image: Gets or sets the image that is displayed on a button control.

Syntax: instance.Image = *value*

Instance: It contains name of the control with which this property is applied.

The default value of Image property is a null reference (**Nothing** in Visual Basic).

4) TextAlign: Gets or sets the alignment of the text on the button control.

Syntax: instance.TextAlign = *value*

Instance: It contains name of the control with which this property is applied.

The default is **MiddleCenter**.

Member name	Description
-------------	-------------

TopLeft	Content is vertically aligned at the top, and horizontally aligned on the left.
TopCenter	Content is vertically aligned at the top, and horizontally aligned at the center.
TopRight	Content is vertically aligned at the top, and horizontally aligned on the right.
MiddleLeft	Content is vertically aligned in the middle, and horizontally aligned on the left.
MiddleCenter	Content is vertically aligned in the middle, and horizontally aligned at the center.
MiddleRight	Content is vertically aligned in the middle, and horizontally aligned on the right.
BottomLeft	Content is vertically aligned at the bottom, and horizontally aligned on the left.
BottomCenter	Content is vertically aligned at the bottom, and horizontally aligned at the center.
BottomRight	Content is vertically aligned at the bottom, and horizontally aligned on the right.

Events:

- 1) Click: Occurs when the control is clicked.

Label Control: Windows Forms Label controls are used to display text or images that cannot be edited by the user. Because the Label control cannot receive the focus, it can also be used to create access keys for other controls. An access key allows a user to select the other control by pressing the ALT key with the access key

Properties:

- 1) Autosize: Gets or sets a value indicating whether the control is automatically resized to display its entire contents.

Syntax: instance.AutoSize = *value*

Instance: It contains name of the control with which this property is applied.

When value is set to **true** then the control adjusts its width to closely fit its contents; otherwise set to **false**. The default is **true**.

- 2) Flatstyle: Gets or sets the flat style appearance of the label control.

Syntax: instance.FlatStyle = *value*

Instance: It contains name of the control with which this property is applied.

Following table shows different FlatStyle values. The default value is **Standard**.

Member name	Description
-------------	-------------

Flat	The control appears flat.
Popup	A control appears flat until the mouse pointer moves over it, at which point it appears three-dimensional.
Standard	The control appears three-dimensional.
System	The appearance of the control is determined by the user's operating system.

3) **Text**: Gets or sets the text associated with this control.

Syntax: instance.Text = *value*

Instance: It contains name of the control with which this property is applied.

Text: It contains text associated with this control.

4) **TextAlign**: Gets or sets the alignment of the text on the button control.

Syntax: instance.TextAlign = *value*

Instance: It contains name of the control with which this property is applied.

Following table shows different TextAlign values. The default is **MiddleCenter**.

Member name	Description
TopLeft	Content is vertically aligned at the top, and horizontally aligned on the left.
TopCenter	Content is vertically aligned at the top, and horizontally aligned at the center.
TopRight	Content is vertically aligned at the top, and horizontally aligned on the right.
MiddleLeft	Content is vertically aligned in the middle, and horizontally aligned on the left.
MiddleCenter	Content is vertically aligned in the middle, and horizontally aligned at the center.
MiddleRight	Content is vertically aligned in the middle, and horizontally aligned on the right.
BottomLeft	Content is vertically aligned at the bottom, and horizontally aligned on the left.
BottomCenter	Content is vertically aligned at the bottom, and horizontally aligned at the center.
BottomRight	Content is vertically aligned at the bottom, and horizontally aligned on the right.

TextBox Control: Windows Forms text boxes are used to get input from the user or to display text. The **TextBox** control is generally used for editable text, although it can also be made read-only. Text boxes can display multiple lines, wrap text to the size of the control, and add basic formatting. The **TextBox** control allows a single format for text displayed or entered in the control.

Properties:

- 1) **Maxlength:** Gets or sets the maximum number of characters that can be manually entered into the text box.

Syntax: instance.MaxLength = *value*

Instance: It contains name of the control with which this property is applied.

The default value of Maxlength property is 32767.

- 2) **Multiline:** Gets or sets a value indicating whether this is a multiline TextBox control.

Syntax: instance.Multiline = *value*

Instance: It contains name of the control with which this property is applied.

When value is set to **true** then the control is a multiline TextBox control; otherwise, **false**. The default is **false**.

- 3) **Text:** Gets or sets the text contents of the text box. This is a dependency property.

Syntax: instance.Text = *value*

Instance: It contains name of the control with which this property is applied.

value: It contains text displayed by this control. By default, user can enter up to 2048 characters in a text box. If Multiline property is set to true then user can enter up to 32 KB of text.

- 4) **TabIndex:** Gets or sets a value that determines the order in which elements receive focus when the user navigates through controls by using the TAB key. This is a dependency property.

Syntax: instance.TabIndex = *value*

Instance: It contains name of the control with which this property is applied.

- 5) **PasswordChar:** Gets or sets the character used to mask characters of a password in a single-line TextBox control.

Syntax: instance.PasswordChar = *value*

Instance: It contains name of the control with which this property is applied.

The character used to mask characters entered in a single-line TextBox control.

RadioButton Control: Windows Forms RadioButton controls present a set of two or more mutually exclusive choices to the user. Radio button provides a set of choices from which user can choose one and only one.

Properties:

- 1) Checked: Gets or sets a value indicating whether the control is checked.

Syntax: instance.Checked = *value*

Instance: It contains name of the control with which this property is applied.

Event:

- 1) CheckChanged: Occurs when the value of the Checked property changes.

CheckBox Control: The Windows Forms CheckBox control indicates whether a particular condition is on or off. It is commonly used to present a Yes/No or True/False selection to the user. User can use check box controls in groups to display multiple choices from which the user can select one or more.

They differ in that only one radio button in a group can be selected at a time. With the check box control any number of check boxes may be selected. There is an important difference: when a user selects a radio button, the other radio buttons in the same group cannot be selected as well. In contrast, any number of check boxes can be selected.

Properties:

- 1) CheckState: Gets or sets the state of the CheckBox.

Syntax: instance.CheckState = *value*

Instance: It contains name of the control with which this property is applied.

Member name	Description
Unchecked	The control is unchecked.
Checked	The control is checked.
Indeterminate	The control is indeterminate. An indeterminate control generally has a shaded appearance.

GroupBox Control: Windows Forms GroupBox controls are used to provide an identifiable grouping for other controls. Typically, you use group boxes to subdivide a form by function. For example, you may have an order form that specifies mailing options such as which overnight carrier to use. Grouping all options in a group box gives the user a logical visual cue, and at design time all the controls can be moved easily — when you move the single GroupBox control, all its contained controls move, too.

ComboBox Control: The Windows Forms ComboBox control is used to display data in a drop-down combo box. By default, the ComboBox control appears in two parts: the top part is a text box that allows the user to type a list item. The second part is a list box that displays a list of items from which the user can select one.

Properties:

1) DropDownStyle: Gets or sets a value specifying the style of the combo box.

Syntax: instance.DropDownStyle = *value*

Instance: It contains name of the control with which this property is applied.

Member name	Description
-------------	-------------

Simple	The text portion is editable. The list portion is always visible.
---------------	---

DropDown	The text portion is editable. The user must click the arrow button to display the list portion. This is the default style.
-----------------	--

DropDownList	The user cannot directly edit the txt portion. The user must click the arrow button to display the list portion. The list displays only if <u>AutoCompleteMode</u> is <u>Suggest</u> or <u>SuggestAppend</u> .
---------------------	--

2) Items: Gets an object representing the collection of the items contained in this ComboBox.

Syntax: *value* = instance.Items

Instance: It contains name of the control with which this property is applied.

3) SelectedIndex: Gets or sets the index specifying the currently selected item.

A zero-based index of the currently selected item. A value of negative one (-1) is returned if no item is selected.

Syntax: instance.SelectedIndex = *value*

Instance: It contains name of the control with which this property is applied.

4) Selecteditem: Gets or sets currently selected item in the ComboBox.

Syntax: `instance.SelectedItem = value`

Instance: It contains name of the control with which this property is applied.

5) Count: Gets the number of items in the collection.

Syntax: `value = instance.Count`

Instance: It contains name of the control with which this property is applied.

6) Sorted: Gets or sets a value indicating whether the items in the combo box are sorted.

Syntax: `instance.Sorted = value`

When it is set to true **then the combo box is sorted; otherwise, false. The default is false.**

Instance: It contains name of the control with which this property is applied.

Methods:

1) Add: Adds an item to the list of items for a ComboBox.

Syntax: `returnValue = instance.Add(item)`

Instance: It contains name of the control with which this property is applied.

2) Insert: Inserts an item into the collection at the specified index.

Syntax: `instance.Insert(index, item)`

Index: The zero-based index location where the item is inserted.

Item: An object representing the item to insert.

Instance: It contains name of the control with which this property is applied.

3) Remove: Removes the specified item from the ComboBox.

Syntax: `instance.Remove(value)`

Value: The Object to remove from the list.

Instance: It contains name of the control with which this property is applied.

4) RemoveAt: Removes an item from the ComboBox at the specified index.

Syntax: `instance.RemoveAt(index)`

index : The index of the item to remove.

Instance: It contains name of the control with which this property is applied.

5) Clear: Removes all items from the ComboBox.

Syntax: `instance.Clear()`

Instance: It contains name of the control with which this property is applied.

6) FindString: Finds the first item in the combo box that starts with the specified string.

Syntax: *returnValue* = instance.FindString(s)

The String to search for. The zero-based index of the first item found; returns -1 if no match is found.

Instance: It contains name of the control with which this property is applied.

7) AddRange: Adds an array of items to the list of items for a ComboBox.

Syntax: instance.AddRange(*items*)

Items: An array of objects to add to the list.

Instance: It contains name of the control with which this property is applied.

Events:

1) SelectedIndexChanged: Occurs when the SelectedIndex property has changed.

ListBox Control: A Windows Forms ListBox control displays a list from which the user can select one or more items. If the total number of items exceeds the number that can be displayed, a scroll bar is automatically added to the ListBox control.

Properties:

1) Items: Gets the items of the ListBox.

Syntax: *value* = instance.Items

Instance: It contains name of the control with which this property is applied.

2) SelectedIndex: Gets or sets the zero-based index of the currently selected item in a ListBox

Syntax: instance.SelectedIndex = *value*

Instance: It contains name of the control with which this property is applied.

A zero-based index of the currently selected item. A value of negative one (-1) is returned if no item is selected.

3) selectedIndices: Gets a collection that contains the zero-based indexes of all currently selected items in the ListBox.

Syntax: *value* = instance.SelectedIndices

Instance: It contains name of the control with which this property is applied.

4) SelectedItem: Gets or sets the currently selected item in the ListBox.

UNIT 3: Developing Windows Forms, Exception Handling

Syntax: instance.SelectedItem = *value*

Instance: It contains name of the control with which this property is applied.

5) SelectedItems: Gets a collection containing the currently selected items in the ListBox.

Syntax: *value* = instance.SelectedItems

Instance: It contains name of the control with which this property is applied.

6) SelectionMode: Gets or sets the method in which items are selected in the ListBox.

Syntax: instance.SelectionMode = *value*

Instance: It contains name of the control with which this property is applied.

Following shows following SelectionMode values. The default is **SelectionMode.One**.

Member name	Description
None	No items can be selected.
One	Only one item can be selected.
MultiSimple	Multiple items can be selected.
MultiExtended	Multiple items can be selected, and the user can use the SHIFT, CTRL, and arrow keys to make selections

7) MultiColumn: Gets or sets a value indicating whether the ListBox supports multiple columns.

Syntax: instance.MultiColumn = *value*

Instance: It contains name of the control with which this property is applied.

When it is set to true then ListBox supports multiple columns; otherwise, **false**. The default is **false**.

8) Sorted: Gets or sets a value indicating whether the items in the ListBox are sorted alphabetically.

Syntax: instance.Sorted = *value*

When it is set to **true** then items in the control are sorted; otherwise, **false**. The default is **false**.

Instance: It contains name of the control with which this property is applied.

9) Count: Gets the number of items in the collection.

Syntax: *value* = instance.Count

The number of items in the collection.

Instance: It contains name of the control with which this property is applied.

Methods:

1) Add: Adds an item to the list of items for a ListBox.

Syntax: `returnValue = instance.Add(item)`

Instance: It contains name of the control with which this property is applied.

Item: An object representing the item to add to the collection.

2) Insert: Inserts an item into the collection at the specified index.

Syntax: `instance.Insert(index, item)`

Instance: It contains name of the control with which this property is applied.

index : The zero-based index location where the item is inserted.

item : An object representing the item to insert.

3) FindString: Finds the first item in the list box that starts with the specified string.

Syntax: `returnValue = instance.FindString(s)`

s : The String to search for.

Instance: It contains name of the control with which this property is applied.

The zero-based index of the first item found; returns -1 if no match is found.

4) Addrange: Adds an array of items to the list of items for a ComboBox.

Syntax: `instance.AddRange(items)`

Instance: It contains name of the control with which this property is applied.

items : An array of objects to add to the list.

5) RemoveAt: Removes the item at the specified index within the collection.

Syntax: `instance.RemoveAt(index)`

Instance: It contains name of the control with which this property is applied.

Index: The zero-based index of the integer to remove.

6) Clear: Removes all items from the collection of selected items.

Syntax: `instance.Clear()`

Instance: It contains name of the control with which this property is applied.

Events:

1) SelectedIndexChanged: Occurs when the SelectedIndex property has changed.

PictureBox: The Windows Forms PictureBox control is used to display graphics in bitmap, GIF, JPEG, metafile, or icon format.

Properties:

1) Image: Gets or sets the image that is displayed by PictureBox.

Syntax: instance.Image = *value*

Instance: It contains name of the control with which this property is applied.

2) Sizemode: Indicates how the image is displayed.

Syntax: instance.SizeMode = *value*

Instance: It contains name of the control with which this property is applied.

Member name	Description
Normal	The image is placed in the upper-left corner of the PictureBox. The image is clipped if it is larger than the PictureBox it is contained in.
StretchImage	The image within the PictureBox is stretched or shrunk to fit the size of the PictureBox.
AutoSize	The PictureBox is sized equal to the size of the image that it contains.
CenterImage	The image is displayed in the center if the PictureBox is larger than the image. If the image is larger than the PictureBox, the picture is placed in the center of the PictureBox and the outside edges are clipped.
Zoom	The size of the image is increased or decreased maintaining the size ratio.

Method:

1) Load: Displays an image in the PictureBox.

Syntax: instance.Load(*url*)

url :The path for the image to display in the PictureBox.

Instance: It contains name of the control with which this property is applied.

HScrollBar and VScrollBar Controls: Windows Forms ScrollBar controls are used to provide easy navigation through a long list of items or a large amount of information by scrolling either horizontally or vertically within an application or control.

Properties:

1) Maximum: Gets or sets the upper limit of values of the scrollable range.

Syntax: instance.Maximum = *value*

Instance: It contains name of the control with which this property is applied.

A numeric value. The default value is 100.

2) Minimum: Gets or sets the lower limit of values of the scrollable range.

Syntax: instance.Minimum = *value*

Instance: It contains name of the control with which this property is applied.

A numeric value. The default value is 0.

3) SmallChange: Gets or sets the value to be added to or subtracted from the Value property when the scroll box is moved a small distance.

Syntax: instance.SmallChange = *value*

Instance: It contains name of the control with which this property is applied.

A numeric value. The default value is 1.

4) LargeChange: Gets or sets a value to be added to or subtracted from the Value property when the scroll box is moved a large distance.

Syntax: instance.LargeChange = *value*

Instance: It contains name of the control with which this property is applied.

A numeric value. The default value is 10

5) Value: Gets or sets a numeric value that represents the current position of the scroll box on the scroll bar control.

Syntax: instance.Value = *value*

Instance: It contains name of the control with which this property is applied.

A numeric value that is within the Minimum and Maximum range. The default value is 0.

Event:

1) Scroll: Occurs when the scroll box has been moved by either a mouse or keyboard action.

Method:

FromArgb: Creates a Color structure from the specified 8-bit color values (red, green, and blue).

Syntax: Color.FromArgb(*red, green, blue*)

red :The red component value for the new Color. Valid values are 0 through 255.

green :The green component value for the new Color. Valid values are 0 through 255.

blue :The blue component value for the new Color. Valid values are 0 through 255.

Color Member: Represents an ARGB (alpha, red, green, blue) color.

CheckedListBox Control: The Windows Forms CheckedListBox control extends the ListBox control. It does almost everything that a list box does and also can display a check mark next to items in the list. Other differences between the two controls are that checked list boxes only support DrawMode.Normal; and that checked list boxes can only have one item or none selected. Note that a selected item appears highlighted on the form and is not the same as a checked item. Checked list boxes can have items added at run time using the **String Collection Editor** or their items can added dynamically from a collection at run time, using the Items property.

Properties:

1) Items: Gets the collection of items in this CheckedListBox.

Syntax: *value* = instance.Items

Instance: It contains name of the control with which this property is applied.

2) CheckOnClick: Gets or sets a value indicating whether the check box should be toggled when an item is selected.

Syntax: instance.CheckOnClick = *value*

Instance: It contains name of the control with which this property is applied.

When it is set to true then the check mark is applied immediately; otherwise it is set to **false**. The default is **false**.

3) SelectionMode: Gets or sets a value specifying the selection mode.

Syntax: instance.SelectionMode = *value*

Instance: It contains name of the control with which this property is applied.

Member name	Description
None	No items can be selected.
One	Only one item can be selected.
MultiSimple	Multiple items can be selected.
MultiExtended	Multiple items can be selected, and the user can use the SHIFT, CTRL, and

arrow keys to make selections

Method:

- 1) Add: Adds an item to the list of items for a `CheckedListBox`, specifying the object to add and whether it is checked.

Syntax: `instance.Add(item, isChecked)`

Instance: It contains name of the control with which this property is applied.

Item: An object representing the item to add to the collection.

isChecked : When it is set to **true** then it checks the item; otherwise, **false**.

- 2) Remove: Removes the specified object from the collection.

Syntax: `instance.Remove(value)`

Instance: It contains name of the control with which this property is applied.

value : An object representing the item to remove from the collection.

- 3) SetItemChecked: Sets CheckState for the item at the specified index to **Checked**.

Syntax: `instance.SetItemChecked(index, value)`

Instance: It contains name of the control with which this property is applied.

index : The index of the item to set the check state for.

value : When it is set to **true** then the items are checked; otherwise, **false**.

When a value of **true** is passed, this method sets the `CheckState` value to **Checked**. A value of **false** sets `CheckState` to **Unchecked**.

- 4) GetItemChecked: Returns a value indicating whether the specified item is checked.

Syntax: `instance.GetItemChecked(index)`

Instance: It contains name of the control with which this property is applied.

index : The index of the item. When it is set to **true** if the item is checked; otherwise, **false**.

- 5) Startswith: Determines whether the beginning of this instance matches the specified string.

Syntax: `instance.StartsWith(value)`

value : The String to compare. When it set to **true then value** matches the beginning of this string; otherwise, **false**.

Instance: It contains name of the control with which this property is applied.

Events:

- 1) SelectedIndexChanged: Occurs when the SelectedIndex property has changed.

DateTimePicker Control: The Windows Forms DateTimePicker control allows the user to select a single item from a list of dates or times. When used to represent a date, it appears in two parts: a drop-down list with a date represented in text, and a grid that appears when you click on the down-arrow next to the list.

Properties

- 1) Value: Gets or sets the date/time value assigned to the control.

Syntax: instance.Value = *value*

Instance: It contains name of the control with which this property is applied. The DateTime value assign to the control.

- 2) MinDate: Gets or sets the minimum date and time that can be selected in the control.

Syntax: instance.MinDate = *value*

The minimum date and time that can be selected in the control. The default is 1/1/1753 00:00:00.

Instance: It contains name of the control with which this property is applied.

- 3) MaxDate: Gets or sets the maximum date and time that can be selected in the control.

Syntax: instance.MaxDate = *value*

The maximum date and time that can be selected in the control. The default is 12/31/9998 23:59:59.

Instance: It contains name of the control with which this property is applied.

- 4) Format: Gets or sets the format of the date and time displayed in the control.

Syntax: instance.Format = *value*

Instance: It contains name of the control with which this property is applied.

One of the DateTimePickerFormat values. The default is Long.

Member name	Description
Long	The DateTimePicker control displays the date/time value in the long date format set by the user's operating system.

Short	The DateTimePicker control displays the date/time value in the short date format set by the user's operating system.
Time	The DateTimePicker control displays the date/time value in the time format set by the user's operating system.
Custom	The DateTimePicker control displays the date/time value in a custom format.

5) CustomFormat: Gets or sets the custom date/time format string.

Syntax: instance.CustomFormat = *value*

Instance: It contains name of the control with which this property is applied.

A string that represents the custom date/time format. The default is a null reference (**Nothing** in Visual Basic).

The following table lists all the valid format strings and their descriptions.

Format string	Description
---------------	-------------

d	The one- or two-digit day.
dd	The two-digit day. Single-digit day values are preceded by a 0.
ddd	The three-character day-of-week abbreviation.
dddd	The full day-of-week name.
h	The one- or two-digit hour in 12-hour format.
hh	The two-digit hour in 12-hour format. Single digit values are preceded by a 0.
H	The one- or two-digit hour in 24-hour format.
HH	The two-digit hour in 24-hour format. Single digit values are preceded by a 0.
m	The one- or two-digit minute.
mm	The two-digit minute. Single digit values are preceded by a 0.
M	The one- or two-digit month number.

MM	The two-digit month number. Single digit values are preceded by a 0.
MMM	The three-character month abbreviation.
MMMM	The full month name.
s	The one- or two-digit seconds.
ss	The two-digit seconds. Single digit values are preceded by a 0.
t	The one-letter A.M./P.M. abbreviation (A.M. is displayed as "A").
tt	The two-letter A.M./P.M. abbreviation (A.M. is displayed as "AM").
y	The one-digit year (2001 is displayed as "1").
yy	The last two digits of the year (2001 is displayed as "01").
yyyy	The full year (2001 is displayed as "2001").

Timer Control: Timers are very useful control because they allows you to create periodic events. Timers are no longer controls but components and they do not appear in window at run time. At design time they appear in component tray. Timers work at behind the screens.

Properties:

- 1) Interval: Gets or sets the time, in milliseconds, before the Tick event is raised relative to the last occurrence of the Tick event.

Syntax: instance.Interval = *value*

Instance: It contains name of the control with which this property is applied.

Value: An Int32 specifying the number of milliseconds before the Tick event is raised relative to the last occurrence of the Tick event. The value cannot be less than one.

To get the number of seconds in the interval, divide this number by 1,000.

Methods:

- 1) Start: Starts the timer.

Syntax: instance.Start()

Instance: It contains name of the control with which this property is applied.

- 2) Stop: Stops the timer.

Syntax: instance.Stop()

Instance: It contains name of the control with which this property is applied.

Events:

- 1) Tick: Occurs when the specified timer interval has elapsed and the timer is enabled.

Working with Advanced Controls

LinkLabel: It represents a Windows label control that can display hyperlinks. The **LinkLabel** control is similar to a Label control with the exception that it can display a hyperlink. Multiple hyperlinks can be specified in the text of the control. Each hyperlink can perform a different task within an application.

Properties:

- 1) **ActiveLinkColor** : Gets or sets the color used to display an active link.

Syntax : instance.ActiveLinkColor = *value*

Instance: It contains name of the control with which this property is applied.

A Color that represents the color to display an active link. The default color is specified by the system is **Color.Red**.

- 2) **DisabledLinkColor** : Gets or sets the color used when displaying a disabled link.

Syntax: instance.DisabledLinkColor = *value*

Instance: It contains name of the control with which this property is applied.

A Color that represents the color when displaying a disabled link. The default is **Empty**.

- 3) **LinkBehavior** : Gets or sets a value that represents the behavior of a link.

Syntax: instance.LinkBehavior = *value*

Instance: It contains name of the control with which this property is applied.

One of the LinkBehavior values. The default is SystemDefault.

Member name	Description
SystemDefault	The behavior of this setting depends on the options set using the Internet Options dialog box in Control Panel or Internet Explorer.
AlwaysUnderline	The link always displays with underlined text.

HoverUnderline	The link displays underlined text only when the mouse is hovered over the link text.
NeverUnderline	The link text is never underlined. The link can still be distinguished from other text by use of the <u>LinkColor</u> property of the <u>LinkLabel</u> control.

4) LinkArea : Gets or sets the range in the text to treat as a link.

Syntax: instance.LinkArea = *value*

A LinkArea that represents the area treated as a link.

Instance: It contains name of the control with which this property is applied.

5) LinkColor : Gets or sets the color used when displaying a normal link.

Syntax: instance.LinkColor = *value*

Instance: It contains name of the control with which this property is applied.

A Color that represents the color of text that is clicked to navigate to a child table. The default is HotTrack.

6) VisitedLinkColor : Gets or sets the color used when displaying a link that that has been previously visited.

Syntax: instance.VisitedLinkColor = *value*

Instance: It contains name of the control with which this property is applied.

A Color that represents the color used to display a link that has been visited. The default value is the user's Internet Explorer setting for the visited link color.

7) LinkVisited : Gets or sets a value indicating whether a link should be displayed as though it were visited.

Syntax: instance.LinkVisited = *value*

Instance: It contains name of the control with which this property is applied.

When it is set to true **then the link has been visited; otherwise, false. The default is false.**

8) Links : Gets the collection of links contained within the LinkLabel.

Syntax: *value* = instance.Links

Instance: It contains name of the control with which this property is applied.

A LinkLabel.LinkCollection that represents the links contained within the LinkLabel control.

Methods:

1) Add : Adds a link to the collection with information to associate with the link.

Syntax:

Links. Add (*start* As Integer, *length* As Integer, *linkData* As Object)

start :The starting character within the text of the label where the link is created.

length:The number of characters after the starting character to include in the link text.

linkData: The object containing the information to associate with the link.

Example:

```
LinkLabel1.Links.Add(14, 11, "info")
```

```
LinkLabel1.Links.Add(33, 4, "www.coriolis.com")
```

2) Remove: Removes the specified link from the collection.

Syntax: Links.Remove (*value*)

value : A LinkLabel.Link that represents the link to remove from the collection.

Events:

LinkClicked : Occurs when a link is clicked inside the link label.

RichTextBox Control: It is an advanced version of TextBox. It provides more formatting option. It can load RTF or TXT format files for reading or editing. RichTextBox allows formatting the text like displaying font types, adding colors, selected text effect, bullets, alignments, indents and so on. By default Rich textbox is multiline. By default maxlength of it is 2147483647.

Properties :

1) SelectedRtf: Gets or sets the currently selected rich text format (RTF) formatted text in the control.

Syntax: instance.SelectedRtf = *value*

Value: The selected RTF text in the control.

Instance: It contains name of the control with which this property is applied.

- 2) BulletIndent: Sets/gets the indentation used in the rich text box when the bullet style is applied to the text.

Syntax : instance.BulletIndent = *value*

Instance: It contains name of the control with which this property is applied.

The number of pixels inserted as the indentation after a bullet. The default is zero.

- 3) ScrollBars : Sets/gets the kind of scroll bars to display in the RichTextBox control.

Syntax : instance.ScrollBars = *value*

Instance: It contains name of the control with which this property is applied.

One of the RichTextBoxScrollBars values. The default is **RichTextBoxScrollBars.Both**.

Member name	Description
None	No scroll bars are displayed.
Horizontal	Display a horizontal scroll bar only when text is longer than the width of the control.
Vertical	Display a vertical scroll bar only when text is longer than the height of the control.
Both	Display both a horizontal and a vertical scroll bar when needed.
ForcedHorizontal	Always display a horizontal scroll bar.
ForcedVertical	Always display a vertical scroll bar.
ForcedBoth	Always display both a horizontal and a vertical scroll bar.

- 4) SelectedText : Sets/gets the selected text within the rich text box.

Syntax : instance.SelectedText = *value*

Instance: It contains name of the control with which this property is applied.

Value: A string that represents the selected text in the control.

5) SelectionAlignment : Sets/gets the alignment to apply to the current selection or insertion point.

Syntax : instance.SelectionAlignment = *value*

Instance: It contains name of the control with which this property is applied.

One of the HorizontalAlignment values.

Member name	Description
Left	The object or text is aligned on the left of the control element.
Right	The object or text is aligned on the right of the control element.
Center	The object or text is aligned in the center of the control element.

6) SelectionBullet : Sets/gets a value specifying if the bullet style is applied to the current selection or insertion point.

Syntax : instance.SelectionBullet = *value*

Instance: It contains name of the control with which this property is applied.

When it is set to true then the current selection or insertion point has the bullet style applied; otherwise, false.

7) SelectionColor : Sets/gets the text color of the current text selection or insertion point.

Syntax : instance.SelectionColor = *value*

Instance: It contains name of the control with which this property is applied.

A Color that represents the color to apply to the current text selection or to text entered after the insertion point.

8) SelectionFont : Sets/gets the font of the current text selection or insertion point.

Syntax : instance.SelectionFont = *value*

Instance: It contains name of the control with which this property is applied.

A Font that represents the font to apply to the current text selection or to text entered after the insertion point.

9) SelectionHangingIndent : Sets/gets the distance between the left edge of the first line of text in the selected paragraph and the left edge of the next lines in the same paragraph.

Syntax : instance.SelectionHangingIndent = *value*

Instance: It contains name of the control with which this property is applied.

The distance, in pixels, for the hanging indent applied to the current text selection or the insertion point.

10) SelectionIndent : Sets/gets the distance in pixels between the left edge of the rich text box and the left edge of the current text selection or text added after the insertion point.

Syntax : instance.SelectionIndent = *value*

Instance: It contains name of the control with which this property is applied.

The current distance, in pixels, of the indentation applied to the left of the current text selection or the insertion point.

11) SelectionLength : Sets/gets the number of characters selected in control.

Syntax : instance.SelectionLength = *value*

Instance: It contains name of the control with which this property is applied.

The number of characters selected in the combo box.

12) SelectionRightIndent : The distance in pixels between the right edge of the RichTextBox

Syntax : instance.SelectionRightIndent = *value*

Instance: It contains name of the control with which this property is applied.

The indentation space, in pixels, at the right of the current selection or insertion point.

Methods :

1) Clear : Clears all text from the text box control.

Syntax : instance.Clear()

Instance: It contains name of the control with which this method is applied.

2) ClearUndo : Clears information about the most recent operation from the undo buffer of the rich text box

Syntax : instance.ClearUndo()

Instance: It contains name of the control with which this method is applied.

3) Copy : Copies the current selection in the rich text box to the Clipboard.

Syntax : instance.Copy()

Instance: It contains name of the control with which this method is applied.

4) Cut : Moves the current selection in the rich text box to the Clipboard.

Syntax : `instance.Cut()`

Instance: It contains name of the control with which this method is applied.

This method will only cut text from the text box if text is selected in the control.

5) Paste : Pastes the contents of the Clipboard into the RichTextBox control.

Syntax : `instance.Paste()`

Instance: It contains name of the control with which this method is applied.

6) Find : Searches for text within the contents of the rich text box.

Syntax : `returnValue = instance.Find(str)`

Instance: It contains name of the control with which this method is applied.

str : The text to locate in the control.

The location within the control where the search text was found or -1 if the search string is not found or an empty search string is specified in the *str* parameter.

7) LoadFile : Loads the contents of a file into the RichTextBox control.

Syntax : `instance.LoadFile(path, fileType)`

Instance: It contains name of the control with which this method is applied.

Path: The name and location of the file to load into the control.

fileType : One of the RichTextBoxStreamType values.

Member name	Description
RichText	A Rich Text Format (RTF) stream.
PlainText	A plain text stream that includes spaces in places of Object Linking and Embedding (OLE) objects.
RichNoOleObjs	A Rich Text Format (RTF) stream with spaces in place of OLE objects. This value is only valid for use with the SaveFile method of the RichTextBox control.
TextTextOleObjs	A plain text stream with a textual representation of OLE objects. This value is only valid for use with the SaveFile method of the RichTextBox control.
UnicodePlainText	A text stream that contains spaces in place of Object Linking and

Embedding (OLE) objects. The text is encoded in Unicode.

8) Redo : Reapplies the last operation that was undone in the RichTextBox control.

Syntax : instance.Redo()

Instance: It contains name of the control with which this method is applied.

9) SaveFile : Saves the contents of the rich text box to a file.

Syntax : instance.SaveFile(*path*, *fileType*)

Instance: It contains name of the control with which this method is applied.

path : The name and location of the file to save.

fileType : One of the RichTextBoxStreamType values.

Member name	Description
RichText	A Rich Text Format (RTF) stream.
PlainText	A plain text stream that includes spaces in places of Object Linking and Embedding (OLE) objects.
RichNoOleObjs	A Rich Text Format (RTF) stream with spaces in place of OLE objects. This value is only valid for use with the SaveFile method of the RichTextBox control.
TextTextOleObjs	A plain text stream with a textual representation of OLE objects. This value is only valid for use with the SaveFile method of the RichTextBox control.
UnicodePlainText	A text stream that contains spaces in place of Object Linking and Embedding (OLE) objects. The text is encoded in Unicode.

10) SelectAll : Selects all text in the rich text box.

Syntax : instance.SelectAll()

Instance: It contains name of the control with which this method is applied.

This method enables you to select all text within the control.

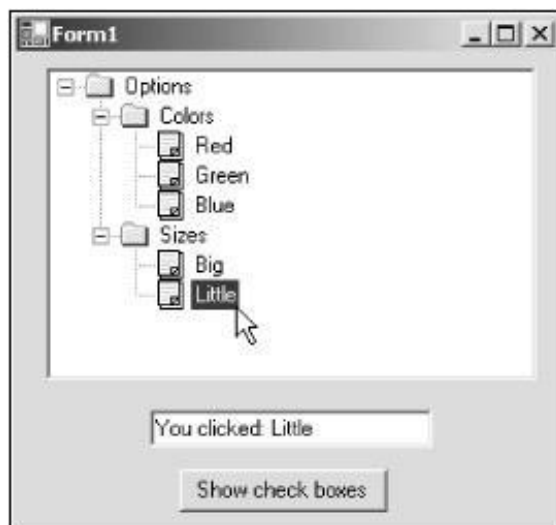
11) Undo : Undoes the last edit operation in the rich text box.

Syntax : instance.Undo()

Instance: It contains name of the control with which this method is applied.

UNIT 3: Developing Windows Forms, Exception Handling

Tree Views: This control is used to display a hierarchy of *nodes*. Each node is not only displayed visually, but also can have child nodes. User can expand and collapse parent nodes by clicking them; when expanded, their children are also visible. You can see a tree view control at work in Figure.

**Properties :**

1) Nodes : Gets the collection of tree nodes that are assigned to the tree view control.

Syntax : `value = instance.Nodes`

Instance: It contains name of the control with which this property is applied.

A `TreeNodeCollection` that represents the tree nodes assigned to the tree view control.

2) CheckBoxes : Gets or sets a value indicating whether check boxes are displayed next to the tree nodes in the tree view control.

Syntax : `instance.CheckBoxes = value`

Instance: It contains name of the control with which this property is applied.

When it is set to **true** then check box is displayed next to each tree node in the tree view control; otherwise, **false**. The default is **false**.

3) ImageIndex : Gets or sets the image-list index value of the default image that is displayed by the tree nodes.

Syntax : `instance.ImageIndex = value`

Instance: It contains name of the control with which this property is applied.

A zero-based index that represents the position of an Image in an ImageList. The default is zero.

4) ImageList : Gets or sets the ImageList that contains the Image objects used by the tree nodes.

Syntax : instance.ImageList = *value*

Instance: It contains name of the control with which this property is applied.

The ImageList that contains the Image objects used by the tree nodes. The default value is a null reference (**Nothing** in Visual Basic).

5) SelectedImageIndex : Gets or sets the image list index value of the image that is displayed when a tree node is selected.

Syntax : instance.SelectedImageIndex = *value*

Instance: It contains name of the control with which this property is applied.

A zero-based index value that represents the position of an Image in an ImageList.

6) SelectedNode : Gets or sets the tree node that is currently selected in the tree view control.

Syntax : instance.SelectedNode = *value*

Instance: It contains name of the control with which this property is applied.

The TreeNode that is currently selected in the tree view control.

7) ShowLines : Gets or sets a value indicating whether lines are drawn between tree nodes in the tree view control.

Syntax : instance.ShowLines = *value*

Instance: It contains name of the control with which this property is applied.

When it is set to true then lines are drawn between tree nodes in the tree view control; otherwise, **false**. The default is **true**.

8) ShowPlusMinus : Gets or sets a value indicating whether plus-sign (+) and minus-sign (-) buttons are displayed next to tree nodes that contain child tree nodes.

Syntax : instance.ShowPlusMinus = *value*

Instance: It contains name of the control with which this property is applied.

When it is set to true then plus sign and minus sign buttons are displayed next to tree nodes that contain child tree nodes; otherwise, **false**. The default is **true**.

9) FullPath: Gets the path from the root tree node to the current tree node.

Syntax : *value* = instance.FullPath

Instance: It contains name of the control with which this property is applied.

The path from the root tree node to the current tree node.

10) PathSeparator: Gets or sets the delimiter string that the tree node path uses.

Syntax : `instance.PathSeparator = value`

Instance: It contains name of the control with which this property is applied.

The delimiter string that the tree node [TreeNode.FullPath](#) property uses. The default is the backslash character (\).

Methods:

1) CollapseAll : Collapses all the tree nodes.

Syntax : `instance.CollapseAll()`

Instance: It contains name of the control with which this method is applied.

The **CollapseAll** method collapses all the `TreeNode` objects, including all the child tree nodes, that are in the `TreeView` control.

2) ExpandAll : Expands all the tree nodes.

Syntax : `instance.ExpandAll()`

Instance: It contains name of the control with which this method is applied.

The **ExpandAll** method expands all the `TreeNode` objects, including all the child tree nodes, that are in the `TreeView` control.

3) Clear: Removes all tree nodes from the collection.

Syntax : `instance.Clear()`

Instance: It contains name of the control with which this method is applied.

4) Insert: Creates a tree node with the specified text and inserts it at the specified index.

Syntax : `instance.Insert(index, text)`

Instance: It contains name of the control with which this method is applied.

index :The location within the collection to insert the node.

text : The text to display in the tree node.

5) Add: Adds a new tree node with the specified label text to the end of the current tree node collection.

Syntax : `instance.Add(text)`

Instance: It contains name of the control with which this method is applied.

text : The label text displayed by the `TreeNode`.

6) Remove: Removes the specified tree node from the tree node collection.

Syntax : `instance.Remove(node)`

Instance: It contains name of the control with which this method is applied.

node : The `TreeNode` to remove.

7) RemoveAt: Removes a tree node from the tree node collection at a specified index.

Syntax : `instance.RemoveAt(index)`

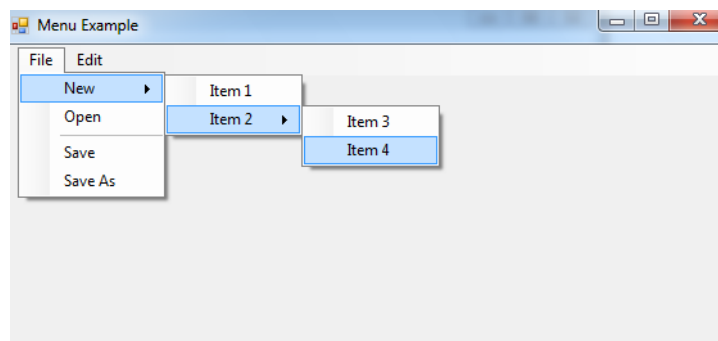
Instance: It contains name of the control with which this method is applied.

index : The index of the `TreeNode` to remove.

Menus: Menus are those controls that allow the user to make selections and also hideaway those selections when they're not needed, saving space in Windows applications, which is always at a premium.

Menus are made up of `MenuItem` objects that represent the individual parts of a menu—menu items can be a parent menu or a menu item in a menu.

Here, there is opened the File menu in the menu bar, and opened a submenu two levels deep.



There are all kinds of options here—you can add submenus to menus that will pop up when the user clicks an arrow in a menu item, display check marks, create menu separators (horizontal bars used in menus to group menu items), assign shortcut keys (like `Ctrl+X`) to menu items. These actions are actually supported by `MenuItem` objects, not `MainMenu` objects.

Creating Menus:

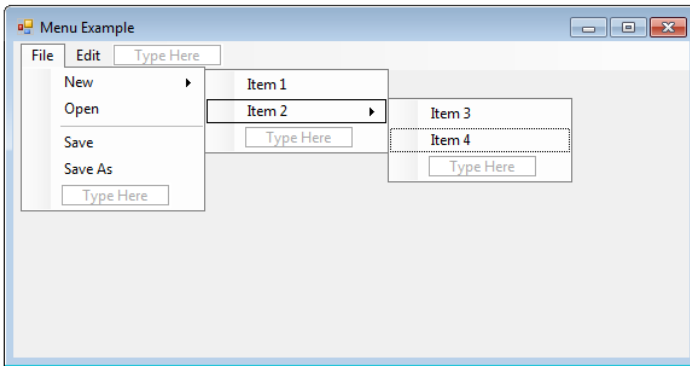
The simplest way to do so is at design time, because all you have to do is to add a MainMenu control from the toolbox to a Windows form. When you do so, the MainMenu control appears in the component tray under the form designer.

To create a new menu, you just have to double-click that text to open a text box which you can use to enter the caption for menus and menu items. When you're creating a new menu item, "Type Here" boxes appear in all the other places you can enter text. To create a new menu in the menu bar, add a submenu to the current menu item, and so on.

Using this control all you have to do is to enter text in the "Type Here" boxes and double-click the resulting menus and menu items to add code to their Click events in the corresponding code designer. To drag menu items around, repositioning them after you've given them a caption, just use the mouse.

You also can create submenus, which involves giving menu items to menu items. When a menu item has a submenu, a right pointing arrow appears in that menu item at run time. Clicking that arrow opens the submenu, displaying additional menu items. And submenus can have submenus, which can have other submenus, and so on.

It's easy to create submenus—you just create the menu item you want to add a submenu to, then select it in a form designer. Doing so opens a "Type Here" box to the right of that menu item; you can enter the captions for the submenu items. Selecting the first item in the submenu opens a "Type Here" box for the next item under it, as well as another "Type Here" box for a new submenu to the right of it. All you have to do is to enter the caption of the submenu items you want, then double-click them to open their Click event in the matching code designer.



MenuStrip1

Adding Checkmarks to Menu Items

You can also add a checkmark to a menu item, which you usually use to indicate to the user that a specific option has been selected. To add a checkmark to a menu item at design time, just click to the left of its caption.

You can use the `Checked` property of a `MenuItem` object to toggle the checkmark; `True` means the checkmark is displayed, `False` means it's hidden.

```
MenuItem7.Checked = Not MenuItem7.Checked
```

Creating Menu Access Keys

Access keys make it possible to select menu items from the keyboard using the `Alt` key. For example, if you make the "F" in the File menu's caption an access key and the "x" in the Exit menu item an access key, the user can select the Exit item by typing `Alt+F` to open the File menu, then `Alt+X` to select the Exit item.

To give an item an access key, you precede the access key in its caption with an ampersand (&). In this example, that means using the captions "&File" and "E&xit". Access keys are underlined in menu captions.

Creating Menu Shortcuts

You can create shortcuts for menu items, which are key combinations that, when pressed, will select that item, making its Click event happen. You set a shortcut with the Shortcut property. To display the shortcut next to the menu item's caption at run time, you set the ShowShortcut property to True (it's True by default).

For example, The Exit item in the File menu a shortcut of Ctrl+X. When the user presses Ctrl+X, this item is activated. To assign a shortcut key combination to a menu item at design time, just select a shortcut key combination from the list that appears when you select the Shortcut property of any menu item in the properties window. At run time, you can use members of the Shortcut enumeration to do the same thing, as in this code:

```
menuItem1.Shortcut = Shortcut.CtrlX
```

Property:

Shortcut: Gets or sets a value indicating the shortcut key associated with the menu item.

Syntax: *instance.Shortcut = value*

Instance: It contains name of the control with which this property is applied.

Creating Menu Separators

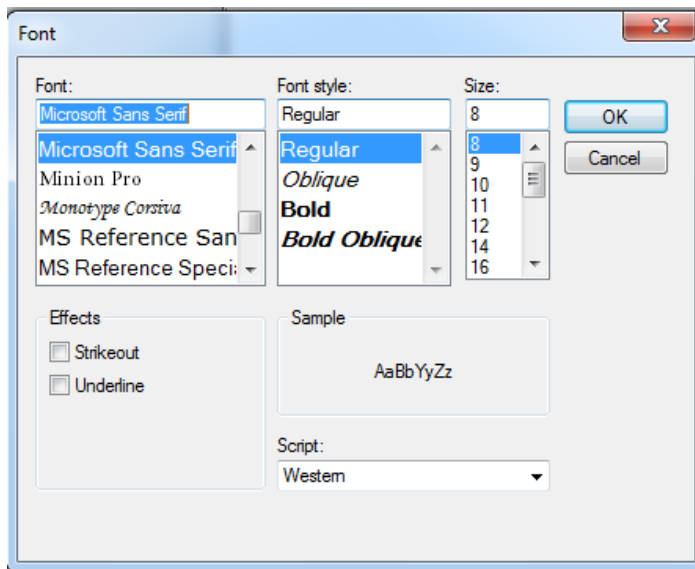
The line is a menu separator, and you can use them to separate the items in a menu into functional groupings. To create a menu separator, you can assign a single hyphen (-) to a menu item's Text property (just type a hyphen in the "Type Here" box when you're creating the menu item).

Disabling Menu Items

To disable, or "gray out" a menu item, so that it can't be selected, you set its Enabled property to False. For example:

```
MenuItem5.Enabled = False
```

Font Dialogs: Font dialogs allow the user to select a font size, face, color, and so on. You can see a font dialog box at work in following Figure.



To display the font dialog box, call the ShowDialog method. This dialog shows list boxes for Font, Style, and Size, checkboxes for effects like Strikeout and Underline, a drop-down list for Script (Script refers to different character scripts that are available for a given font—for example, Hebrew), and a sample of how the font will appear. You can recover these settings using properties of the same names of the Font object returned by the Font property.

Properties:

1) Font: Gets or sets the selected font.

Syntax: *value* = instance.Font

Instance: It contains name of the control with which this property is applied.

2) Color : Gets/sets the selected font color.

Syntax : instance.Color = *value*

Instance: It contains name of the control with which this property is applied.

The color of the selected font. The default value is Black.

3) MaxSize : Gets/sets the maximum point size a user can select.

Syntax : instance.MaxSize = *value*

Instance: It contains name of the control with which this property is applied.

The maximum point size a user can select. The default is 0.

4) MinSize : Gets/sets the minimum point size a user can select.

Syntax : instance.MinSize = *value*

Instance: It contains name of the control with which this property is applied.

The minimum point size a user can select. The default is 0.

5) ShowColor : Gets/sets whether the dialog box displays the color choice.

Syntax : instance.ShowColor = *value*

Instance: It contains name of the control with which this property is applied.

When it is set to true then the dialog box displays the color choice; otherwise, **false**. The default value is **false**.

6) ShowEffects : Gets/sets whether the dialog box contains controls that allow the user to specify strikethrough, underline, and text color options.

Syntax : instance.ShowEffects = *value*

Instance: It contains name of the control with which this property is applied.

When it is set to true then the dialog box contains controls to set strikethrough, underline, and text color options; otherwise, **false**. The default value is **true**.

Methods:

1) ShowDialog : Runs a common dialog box with a default owner.

Syntax : *returnValue* = instance.ShowDialog()

Instance: It contains name of the control with which this method is applied.

DialogResult.OK if the user clicks OK in the dialog box; otherwise, DialogResult.Cancel.

DialogResult

Member name	Description
----------------	-------------

None	Nothing is returned from the dialog box. This means that the modal dialog continues running.
-------------	---

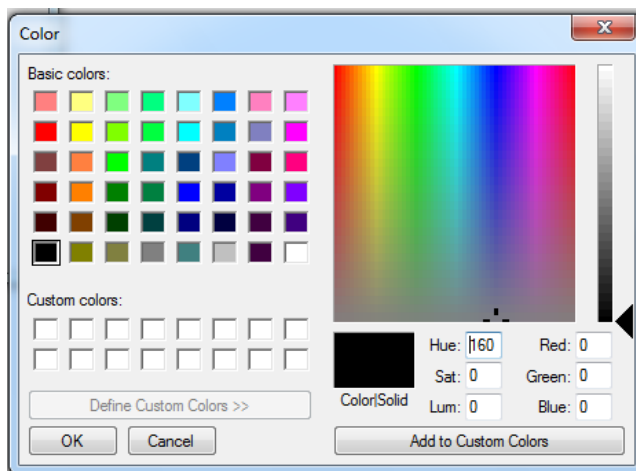
OK	The dialog box return value is OK (usually sent from a button labeled OK).
-----------	---

Cancel	The dialog box return value is Cancel (usually sent from a button labeled Cancel).
---------------	---

Abort	The dialog box return value is Abort (usually sent from a button labeled Abort).
--------------	---

Retry	The dialog box return value is Retry (usually sent from a button labeled Retry).
Ignore	The dialog box return value is Ignore (usually sent from a button labeled Ignore).
Yes	The dialog box return value is Yes (usually sent from a button labeled Yes).
No	The dialog box return value is No (usually sent from a button labeled No).

Color Dialogs: Color dialogs allow the user to select a color in an easy way. The principal property you use of these dialogs is the Color property, which returns a Color object, ready for use. You can see a color dialog box in following Figure



In Figure, I've opened the color dialog fully (by clicking Define Custom Colors) to let the user define their own colors with color values and hue, saturation, and luminosity. **Properties:**

- 1) AllowFullOpen : Gets or sets a value indicating whether the user can use the dialog box to define custom colors.

Syntax : instance.AllowFullOpen = *value*

Instance: It contains name of the control with which this property is applied.

When it is set to true then the user can define custom colors; otherwise, **false**. The default is **true**.

- 2) AnyColor : Gets or sets a value indicating whether the dialog box displays all available colors in the set of basic colors.

Syntax : `instance.AnyColor = value`

Instance: It contains name of the control with which this property is applied.

When it is set to true then the dialog box displays all available colors in the set of basic colors; otherwise, **false**. The default value is **false**.

3) Color :Gets or sets the color selected by the user.

Syntax : `instance.Color = value`

Instance: It contains name of the control with which this property is applied.

The color selected by the user. If a color is not selected, the default value is black.

4) CustomColors :Gets or sets the set of custom colors shown in the dialog box.

Syntax : `instance.CustomColors = value`

Instance: It contains name of the control with which this property is applied.

A set of custom colors shown by the dialog box. The default value is a null reference (**Nothing** in Visual Basic).

5) FullOpen :Gets or sets a value indicating whether the controls used to create custom colors are visible when the dialog box is opened

Syntax : `instance.FullOpen = value`

Instance: It contains name of the control with which this property is applied.

When it is set to true then the custom color controls are available when the dialog box is opened; otherwise, **false**. The default value is **false**.

6) SolidColorOnly :Gets or sets a value indicating whether the dialog box will restrict users to selecting solid colors only.

Syntax : `instance.SolidColorOnly = value`

Instance: It contains name of the control with which this property is applied.

When it is set to true then users can select only solid colors; otherwise, **false**. The default value is **false**.

Methods: ShowDialog : this method is same for all dialog control.

OpenFileDialog Control: The Windows Forms OpenFileDialog component is a pre-configured dialog box. It is the same **Open File** dialog box exposed by the Windows operating system.

By relying on standard Windows dialog boxes, you create applications whose basic functionality is immediately familiar to users. Be aware, however, that when using the OpenFileDialog component, you must write your own file-opening logic.

When it is added to a form, the OpenFileDialog component appears in the tray at the bottom of the Windows Forms Designer.

Properties:

- 1) MultiSelect: Gets or sets a value indicating whether the dialog box allows multiple files to be selected.

Syntax: `instance.Multiselect = value`

Instance: It contains name of the control with which this property is applied.

When it is set to true then the dialog box allows multiple files to be selected together or concurrently; otherwise, **false**. The default value is **false**.

- 2) CheckFileExists: Gets or sets a value indicating whether the dialog box displays a warning if the user specifies a file name that does not exist.

Syntax: `instance.CheckFileExists = value`

Instance: It contains name of the control with which this property is applied.

When it is set to true then dialog box displays a warning when the user specifies a file name that does not exist; otherwise, **false**. The default value is **true**.

- 3) Filter: Gets or sets the current file name filter string, which determines the choices that appear in the "Save as file type" or "Files of type" box in the dialog box.

Syntax: `instance.Filter = value`

Instance: It contains name of the control with which this property is applied.

Value : The file filtering options available in the dialog box.

For each filtering option, the filter string contains a description of the filter, followed by the vertical bar (|) and the filter pattern. The strings for different filtering options are separated by the vertical bar.

The following is an example of a filter string:

`Text files (*.txt)|*.txt|All files (*.*)|*.*`

4) **Filename:** Gets the file names of all selected files in the dialog box.

Syntax: `value = instance.FileNames`

Instance: It contains name of the control with which this property is applied.

Filenames: An array of type String, containing the file names of all selected files in the dialog box.

5) **Title:** Gets or sets the file dialog box title.

Syntax: `instance.Title = value`

Instance: It contains name of the control with which this property is applied.

The file dialog box title. The default value is an empty string ("").

6) **InitialDirectory:** Gets or sets the initial directory displayed by the file dialog box.

Syntax: `instance.InitialDirectory = value`

Instance: It contains name of the control with which this property is applied.

The initial directory displayed by the file dialog box. The default is an empty string ("").

SaveFileDialog Control: The Windows Forms SaveFileDialog component is a pre-configured dialog box. It is the same as the standard **Save File** dialog box used by Windows. Use it as a simple solution for enabling users to save files instead of configuring your own dialog box. By relying on standard Windows dialog boxes, the basic functionality of applications you create is immediately familiar to users. Be aware, however, that when using the SaveFileDialog component, you must write your own file-saving logic.

Properties:

1) **DefaultExt:** Gets or sets the default file name extension.

Syntax: `instance.DefaultExt = value`

Instance: It contains name of the control with which this property is applied.

The default file name extension. The returned string does not include the period. The default value is an empty string ("").

Error Handling:

There are two types of handling errors that occur at run time in VB .NET

- 1) Structured exception handling.
- 2) Unstructured exception handling.

What's an exception?

Exceptions are just runtime errors; in Visual Basic, the terms *exception handling* and *error handling* have become inter-changeable. Exceptions occur when a program is running (as opposed to syntax errors, which will prevent VB.NET from running your program at all). You can trap such exceptions and recover from them, rather than letting them bring your program to an inglorious end.

1) Unstructured Exception Handling

The old error-handling mechanism in VB6 and before is now called unstructured exception handling, and it revolves around the On Error Goto statement. You use this statement to tell VB .NET where to transfer control to in case there's been an exception.

On Error GoTo statement: The On Error GoTo statement enables exception handling and specifies the location of the exception- handling code within a procedure.

Syntax: On Error { GoTo [*line* | 0 | -1] | Resume Next }

GoTo *line*—Enables the exception-handling code that starts at the line specified in the required *line* argument. The *line* argument is any line label or line number. If an exception occurs, program execution goes to the given location. (Note that the specified line must be in the same procedure as the On Error statement.)

GoTo 0—Disables enabled exception handler in the current procedure and resets it to Nothing.

GoTo -1—Same as GoTo 0.

Resume Next—Specifies that when an exception occurs, execution skips over the statement that caused the problem and goes to the statement immediately following. Execution continues from that point.

Example:

```
Private Sub Form1_Load()  
    Dim int1 = 0, int2 = 1, int3 As Integer  
    On Error GoTo Handler  
    int3 = int2 / int1  
    Exit Sub  
Handler:  
    MsgBox ("Overflow error")  
End Sub
```

Using Resume Next and Resume Line

One of the most useful aspects of unstructured exception handling is the Resume statement, which lets you resume program execution even after an exception has occurred. You can use Resume to resume execution with the statement that caused the exception, Resume Next to resume execution with the statement after the one that caused the exception, and Resume *line*, where *line* is a line number or label that specifies where to resume execution.

Example:

```
Private Sub Form1_Load()  
    Dim int1 = 0, int2 = 1, int3 As Integer  
    On Error GoTo Handler  
    int3 = int2 / int1  
    MsgBox("Program completed...")  
    Exit Sub  
Handler:  
    If (TypeOf Err.GetException() Is OverflowException) Then  
        MsgBox("Overflow error!")  
        Resume Next  
    End If  
End Sub
```


Using On Error GoTo 0

To turn off unstructured exception handling, you can use the On Error GoTo 0 or On Error GoTo -1 statements.

Err Object: Contains information about run-time errors.

Properties:

- 1) Number: Gets or sets a numeric value that specifies an error.
- 2) Description: Returns or sets a **String** expression containing a descriptive string associated with an error.

Example:

```
Private Sub Form1_Load()  
    Dim int1 = 0, int2 = 1, int3 As Integer  
    On Error GoTo Handler  
    int3 = int2 / int1  
    MsgBox("Program completed...")
```

Handler:

```
    MsgBox("Error number {0} occurred: {1}", Err.Number, Err.Description)
```

End Sub

Here's what you see when you run this application:

Error number 6 occurred: Exception of type System.OverflowException was thrown.

2) Structured Exception Handling

Structured exception handling is based on a particular statement, the Try...Catch...Finally statement, which is divided into a Try block, optional Catch blocks, and an optional Finally block. The Try block contains code where exceptions can occur, the Catch block contains code to handle the exceptions that occur. If an exception occurs in the Try block, the code *throws* the exception—actually an object based on the Visual Basic Exception class—so it can be caught and handled by the appropriate Catch statement. After the rest of the statement finishes, execution is always passed to the Finally block, if there is one.

Syntax:

Try

 [*tryStatements*]

 [Catch [*exception1* [As *type1*]] [When *expression1*]

catchStatements1

 [Exit Try]]

 [Catch [*exception2* [As *type2*]] [When *expression2*]

catchStatements2

 [Exit Try]]

 [Catch [*exceptionn* [As *typen*]] [When *expressionn*]

catchStatementsn]

 [Exit Try]]

 [Finally

 [*finallyStatements*]

End Try

Try—Begins the Try block for structured exception handling.

tryStatements—Sensitive statements where you expect exceptions.

Catch—If an exception happens in the Try block, the exception is thrown and each Catch statement is examined in order to determine if it will handle the exception.

exception—A variable name you give the exception. The value of *exception* is the value of the thrown exception.

type—Specifies the type of the exception you're catching in a Catch statement.

When—A Catch clause with a When clause will only catch exceptions when *expression* evaluates to True.

expression—An expression used to select exceptions to handle; must be convertible to a Boolean value Often used to filter exceptions by number.

catchStatements—Statements to handle exceptions occurring in the Try block.

Exit Try—Statement that breaks out of the Try...Catch...Finally structure. Execution is transferred to the code immediately following the End Try statement. Note that Exit Try is not allowed in Finally blocks.

Finally—A Finally block is always executed when execution leaves any part of the Try statement.

finallyStatements—Statements that are executed after all other exception processing has occurred.

Example :

```
Private Sub Form1_Load()  
    Dim int1 = 0, int2 = 1, int3 As Integer  
    Try  
        int3 = int2 / int1  
        MsgBox("The answer is {0}", int3)  
    Catch  
        MsgBox("Exception: Arithmetic overflow!")  
    End Try  
End Sub
```

Exception Filtering in the Catch Block

When you're handling exceptions, you usually want to handle different types of exceptions differently, according to the nature of the exception that occurred. This process is called *filtering*. There are actually two ways to filter exceptions with Catch blocks. First, you can filter on specific classes of exceptions, which means you have to prepare for the various exceptions you want to handle.

Example:

```
Private Sub Form1_Load()  
    Dim int1 = 0, int2 = 1, int3 As Integer  
    Try  
        int3 = int2 / int1
```

```
MsgBox("The answer is {0}", int3)
Catch e1 As OverflowException
    MsgBox("Exception: Arithmetic overflow!")
End Try
End Sub
```

Using Multiple *Catch* Statements

You also can use multiple Catch statements when you filter exceptions. Here's an example that specifically handles overflow, invalid argument, and argument out of range exceptions:

Example:

```
Private Sub Form1_Load()
    Dim int1 = 0, int2 = 1, int3 As Integer
    Try
        int3 = int2 / int1
        MsgBox("The answer is {0}", int3)
    Catch e1 As System.OverflowException
        MsgBox("Exception: Arithmetic overflow!")
    Catch e1 As System.ArgumentException
        MsgBox("Exception: Invalid argument value!")
    End Try
End Sub
```

If you want to add a general exception handler to catch any exceptions not filtered, you can add a Catch block for the Exception class at the end of the other Catch blocks:

Example:

```
Private Sub Form1_Load()
    Dim int1 = 0, int2 = 1, int3 As Integer
    Try
        int3 = int2 / int1
        MsgBox("The answer is {0}", int3)
    Catch e1 As System.OverflowException
```

```
MsgBox("Exception: Arithmetic overflow!")
Catch e1 As System.ArgumentException
    MsgBox("Exception: Invalid argument value!")
Catch e1 As Exception
    MsgBox("Exception occurred!")
End Try
End Sub
```

Using *Finally*

The code in the Finally block, if there is one, is always executed in a Try...Catch...Finally statement, even if there was no exception, and even if you execute an Exit Try statement. This allows you to deallocate resources and so on.

Example:

```
Private Sub Form1_Load()
    Dim int1 = 0, int2 = 1, int3 As Integer
    Try
        int3 = int2 / int1
        MsgBox("The answer is {0}", int3)
    Catch e1 As System.OverflowException
        MsgBox("Exception: Arithmetic overflow!")
    Catch e1 As System.ArgumentException
        MsgBox("Exception: Invalid argument value!")
    Finally
        MsgBox("Execution of sensitive code is complete")
    End Try
End Sub
```

Output :

Exception: Arithmetic overflow!

Execution of sensitive code is complete

Throwing an Exception

You can throw an exception using the Throw statement, and you can also rethrow a caught exception using the Throw statement. Here's an example where I'm explicitly throwing an overflow exception:

Example:

```
Private Sub Form1_Load()  
    Try  
        Throw New OverflowException()  
    Catch e1 As Exception  
        MsgBox(e1.Message)  
    End Try  
End Sub
```

Throwing a Custom Exception

You can customize the exceptions you throw by creating a new exception object based on the ApplicationException object. Here's an example where I'm creating a new ApplicationException object with the text "This is a new exception", and then throwing and catching that exception:

Example:

```
Private Sub Form1_Load()  
    Try  
        Throw New ApplicationException("This is a new exception")  
    Catch e1 As Exception  
        MsgBox (e1.Message)  
    End Try  
End Sub
```

Output :

This is a new exception